Synopsis

House Listing Model

Problem Id: 5

Student Name: Ashu Khanduri

University Roll number: 2028673

Section: K

Student Name: Srishti Nahar

University Roll number: 2028187

Section: K

Student Name: Anushka Pun

University Roll number: 2028656

Section: K

● **Problem Description:**

Design and develop a regression-based machine learning system to predict house sale price using a structured real-estate dataset consisting of 25 property-related features. The dataset represents residential housing information collected from different regions and includes both numerical and categorical attributes that influence property valuation.

The dataset contains the following 25 input features: property area (square feet), number of bedrooms, number of bathrooms, number of floors, year built, property age, renovation status, lot size, distance to city center (km), neighborhood quality score, crime rate index, nearby school rating, hospital proximity (km), shopping center proximity (km), public transport accessibility score, parking availability (binary), property type (categorical), construction quality rating, energy efficiency score, water supply reliability score, electricity supply reliability score, internet availability score, green space availability index, flood risk index, and noise pollution level.

The project should begin with Exploratory Data Analysis (EDA) to analyze feature distributions, correlations, and their relationship with the target variable. Perform data preprocessing including handling missing values, encoding categorical features, and normalization or feature scaling.

Develop and train suitable regression models to predict house prices and visualize predicted versus actual values as well as residuals. Finally, evaluate the trained models using appropriate regression performance metrics such as R2 score and error measures such as MAE, MSE, and analyze the impact of different features on the prediction outcome. The project should demonstrate a complete machine learning pipeline from raw data to model evaluation and interpretation.

● **Steps of Implementation:**

1. Problem Definition

This step involves clearly defining the objective of the project, which is to predict house sale prices based on various property features. Since the output is a continuous value, it is a regression problem under supervised learning. You also identify input variables (features) and the target variable (price). A clear problem definition helps guide model selection and evaluation criteria.

2. Data Collection & Loading

In this step, the dataset is gathered and loaded into the working environment using tools like Pandas. The structure of the dataset is examined, including the number of rows, columns, and data types. Initial checks help ensure the dataset is correctly loaded and ready for analysis. It also helps identify any obvious inconsistencies or issues.

3. Exploratory Data Analysis (EDA)

EDA is performed to understand the data's structure and patterns. Statistical summaries and visualizations are used to analyze distributions, detect outliers, and observe relationships between features and the target variable. Correlation analysis helps identify which features strongly influence house prices. This step builds intuition and guides preprocessing and model selection.

4. Data Preprocessing

This step prepares the raw data for modeling by cleaning and transforming it. Missing values are handled using appropriate techniques such as mean or mode imputation. Categorical variables are converted into numerical form using methods like One-Hot Encoding. Feature scaling, such as Standardization, ensures all variables are on a comparable scale.

5. Feature & Target Split

The dataset is divided into input features (X) and the target variable (y), which is the house price. This separation is necessary because the model learns patterns from input features to predict the target. It clearly defines what data is used for training versus what is being predicted. Proper splitting ensures organized data flow in later steps.

6. Train-Test Split

The dataset is split into training and testing subsets and the training set is used to build the model, while the test set evaluates its performance on unseen data. This helps detect overfitting and ensures the model generalizes well. It is a crucial step for reliable performance assessment.

7. Model Development

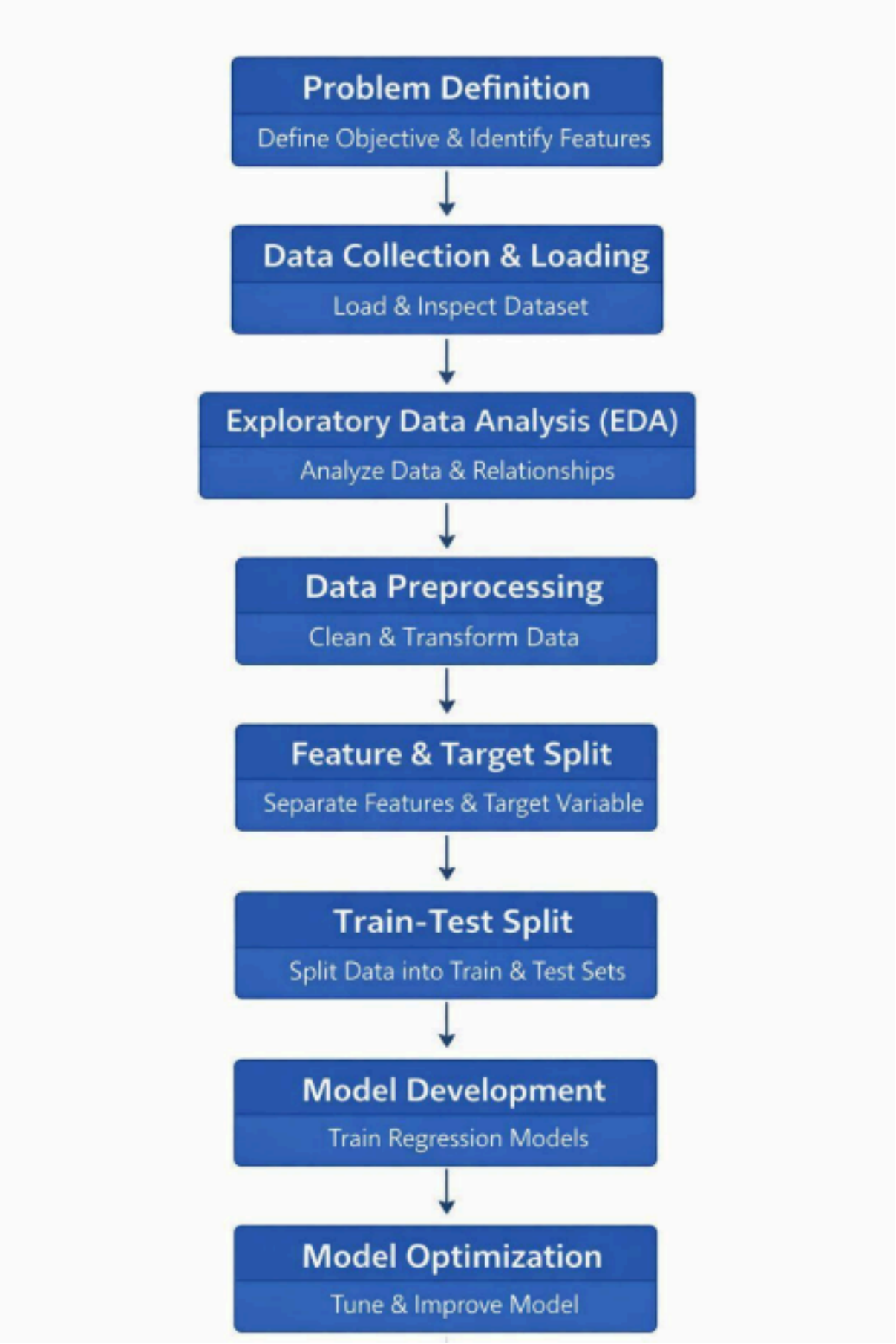
In this step, different regression algorithms such as Linear Regression, Decision Trees, and Random Forest are trained on the data. Each model learns patterns between input features and house prices. Trying multiple models allows comparison of performance and selection of the best one. This step forms the core predictive component of the system.

8. Model Evaluation

The trained model is evaluated using performance metrics like R2 Score, Mean Absolute Error, and Mean Squared Error. These metrics quantify how accurately the model predicts house prices. Evaluation on test data ensures unbiased performance measurement. It helps in comparing models and identifying improvements.

9. Model Optimization

Model performance is improved by tuning hyperparameters using techniques like grid search or cross-validation. This process finds the best combination of model settings for optimal results. Optimization reduces errors and enhances generalization ability. It ensures the model performs well on unseen data.


Problem Definition
Define Objective & Identify Features
Data Collection & Loading
Load & Inspect Dataset
Exploratory Data Analysis (EDA)
Analyze Data & Relationships
Data Preprocessing
Clean & Transform Data
Feature & Target Split
Separate Features & Target Variable
Train-Test Split
Split Data into Train & Test Sets
Model Development
Train Regression Models
Model Optimization
Tune & Improve Model

- **Expected Modules/ Libraries to be used :**

Core Data Manipulation

1. NumPy

NumPy is the foundational library for numerical computing in Python, centered around the powerful N-dimensional array object. It provides high-performance mathematical functions to operate on these arrays, making it much faster than standard Python lists. It's the engine that powers most other data science libraries.

2. Pandas

Pandas is built on top of NumPy and introduces the DataFrame, a 2D table-like structure similar to an Excel spreadsheet. It is the go-to tool for data cleaning, manipulation, and analysis. It allows you to easily handle missing data, merge datasets, and filter information using intuitive labels.

Visualization Tools

3. Matplotlib

Matplotlib is the grandfather of Python visualization, offering a low-level interface for creating static, animated, and interactive plots. It gives you total control over every element of a figure, from line styles to axis labels.

4. Seaborn

Seaborn is a high-level library built on Matplotlib that specializes in making beautiful, statistical graphics with less code. It integrates seamlessly with Pandas DataFrames and comes with built-in themes and color palettes. It excels at complex visualizations like heatmaps, violin plots, and time-series distributions.

Scikit-Learn (ML Workflow)

5. sklearn.preprocessing

This module is used to transform raw data into a format suitable for machine learning models. It includes tools for Scaling (making sure all numbers are on the same scale), Encoding (converting text labels into numbers), and handling outliers. Proper preprocessing often determines whether a model succeeds or fails.

6. sklearn.linear_model

This contains algorithms that assume a linear relationship between input variables and the target output. Its most famous members are Linear Regression (for predicting continuous numbers) and Logistic Regression (for classification). These models are prized for being fast, easy to interpret, and highly efficient.

7. sklearn.model_selection

This module helps you organize how you train and test your AI. It includes functions like train_test_split to divide your data so you don't cheat by testing on what the model has already seen. It also features Cross-Validation and GridSearch to help find the best possible settings for your model.

8. sklearn.feature_selection

Not all data is useful; sometimes noise can confuse a model. This module provides techniques to select the most relevant variables (features) for your prediction. By removing redundant or irrelevant columns, you can reduce the complexity of your model, speed up training, and improve accuracy.

9. sklearn.metrics

This is the report card for the model. It contains functions to calculate how far off our predictions were from reality.

- **Expected Evaluation metrics and Plots to be used:**

1. Visualizations

Before building the model, visual diagnostics are used to ensure the data is clean and that our assumptions are grounded in reality.

- **Boxplots (Risk & Anomaly Detection):** These plots are used to identify **statistical outliers**. By isolating extreme values (e.g., ultra-luxury properties), we can decide whether to include them in the general model or treat them as a separate market segment. This prevents a few rare cases from distorting the "typical" price predictions.
- **Scatter Plots (Variable Relationships):** We use these to visualize the correlation between independent drivers (like distance) and the target (price). This confirms whether a relationship is linear or requires a more complex mathematical approach to

capture the trend accurately.

- **Actual vs. Predicted Plots (Model Validation):** This is the ultimate "audit" of the model. By plotting our predictions against the real historical values, we can visually confirm if the model is consistently accurate or if it tends to over-estimate or under-estimate in certain price brackets.

2. Quantitative Metrics

These metrics provide an objective scorecard to justify the model's accuracy before it is deployed for business use.

- R-Squared (R^2): The Market Variance Capture
  This represents the percentage of the price fluctuations that our model can explain. A high R^2 indicates that the model has successfully learned the primary factors driving property value.
- Mean Absolute Error (MAE): The Real-World Margin of Error
  This is the most intuitive metric for stakeholders. It represents the average expected difference between our prediction and the actual price in money terms.
- Root Mean Squared Error (RMSE): The Risk Indicator
  While MAE treats all errors equally, RMSE penalizes large misses more heavily. This is a critical safety metric; it ensures the model isn't making rare but massive errors that could lead to significant financial miscalculation.

- **Required topics from the subject:**

1. Exploratory Data Analysis (EDA) & Descriptive Statistics

Before any prediction can happen, the data must be profiled.

- **Distribution Analysis:** Understanding the spread of data using **Mean, Median, and Interquartile Range (IQR)**.
- **Outlier Theory:** Knowing how to identify and handle statistical anomalies that can skew business forecasts.
- **Bivariate Analysis:** The study of how two variables (like "Distance" and "Price") interact, which is the foundation of identifying market drivers.

2. Feature Engineering & Data Pre-processing

This is the most critical part of the code that turns raw data into machine-readable intelligence.

- **Categorical Encoding Strategies:** Specifically **One-Hot Encoding**. This is the process of converting qualitative data (City Names, Neighborhoods) into quantitative binary vectors (1s and 0s).
- **The Dummy Variable Trap:** Understanding why we used drop_first=True to prevent redundant data from confusing the model’s logic.

- **Data Cleaning:** Managing missing values and data type conversion (e.g., ensuring "Price" is a float and not a text string).

3. Predictive Analytics (Regression Modeling)

This domain covers the AI part of the project i.e. the algorithms that learn from history to predict the future.

- **Linear & Non-Linear Regression:** Understanding the mathematical relationship between input features and the target price.
- **Supervised Learning Workflow:** The process of training a model on historical data so it can generalize to new, unseen properties.

4. Model Validation & Performance Diagnostics.

- **Error Quantization (MAE, RMSE):** The ability to translate mathematical variance into real-world business risk and average dollar-value margins of error.
- **Variance Explanation (R^2):** Assessing the goodness of fit to determine if the model is actually capturing the market's behavior or just memorizing noise (Overfitting).
- **Bias-Variance Tradeoff:** Finding the balance between a model that is too simple (underfitting) and one that is too complex (overfitting).

- **Platform Required:**

jupyter notebook

- **Books and Link Sources:**

1. https://www.geeksforgeeks.org/ [understanding theory and concepts] 2. Chatgpt/Gemini : For helping in understanding what to do and why we are doing it 3. Pandas Official Documentation – https://pandas.pydata.org/docs/
4. Scikit-learn Official Documentation- https://scikit-learn.org/docs/